UNIVERSIDAD DON BOSCO

Nombre del módulo:

Desarrollo de Software para Móviles DSM941

Título de la actividad:

Foro 1

Equipo:

Katia María Hernández Pérez HP221350

David Roberto Ferrer Coto FC243112

Melvin Eduardo Robles Rodas RR191220

Juan Carlos Ramírez Chávez RC231487

Fecha Entrega:

26 de abril de 2026

ÍNDICE

Introducción.....3
Contenido.....4
- **Capacidades Técnicas y Aplicaciones Prácticas**.....5
- **Ventajas de Jetpack Compose**.....6
- **Desventajas de Jetpack Compose**.....8

Descripción del flujo implementado y decisiones de diseño.....9
Conclusiones.....10
- **Conclusiones personales**.....10

Referencias.....12
Link de video.....13

Introducción

Jetpack Compose es una herramienta moderna desarrollada por Google para la creación de interfaces de usuario en aplicaciones Android. Su enfoque declarativo permite simplificar el desarrollo, reduciendo la cantidad de código necesario y mejorando la mantenibilidad de las aplicaciones.

En el presente trabajo se realiza una investigación sobre Jetpack Compose, así como el desarrollo de una aplicación móvil que incluye un sistema de login y el cálculo de notas de un estudiante. Este proyecto permite aplicar los conocimientos adquiridos en Kotlin y desarrollo móvil.

Contenido

Jetpack Compose

Es un kit de herramientas moderno para crear UI nativas de Android. Jetpack Compose simplifica y acelera el desarrollo de IU en Android con menos código, herramientas potentes y API intuitivas de Kotlin.

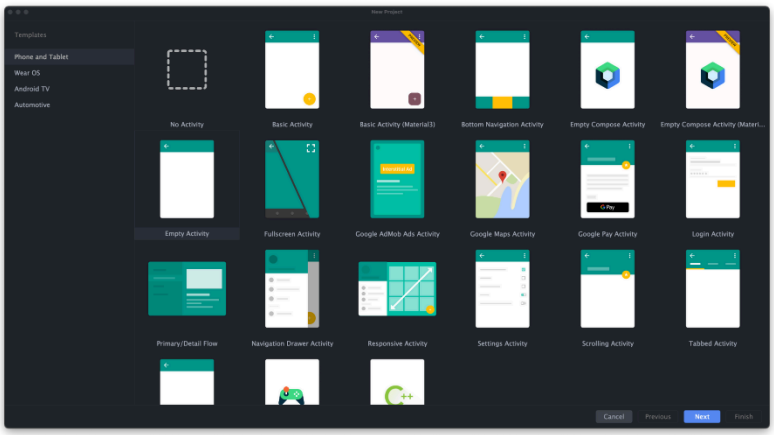
Combina un modelo de programación reactivo con la concisión y facilidad de uso del lenguaje de programación Kotlin. Es totalmente declarativo, lo que significa que describes tu IU si llamas a una serie de funciones que transforman datos en una jerarquía de IU. Cuando los datos subyacentes cambian, el framework vuelve a ejecutar estas funciones automáticamente y actualiza la jerarquía de la IU por ti.

Una app de Compose tiene funciones de compatibilidad, que son solo funciones normales marcadas con @Composable, que pueden llamar a otras funciones de compatibilidad. Una función es todo lo que necesitas para crear un nuevo componente de IU. La anotación le indica a Compose que agregue compatibilidad especial a la función para actualizar y mantener tu IU con el tiempo. Compose te permite estructurar el código en fragmentos pequeños.

Al crear pequeños elementos componibles reutilizables, es fácil crear una biblioteca de elementos de IU que se usan en tu app. Cada uno es responsable de una parte de la pantalla y se puede editar de forma independiente.

Figura 1

Interfaz de seleción de tipo de plantilla en android studio



Nota. Creación propia (2026).

Capacidades Técnicas y Aplicaciones Prácticas

Jetpack Compose simplifica el proceso de desarrollo de una aplicación en Android gracias a su enfoque declarativo, puedes escribir menos código en comparación con el enfoque imperativo actual de creación de vistas, lo que también significa menos errores potenciales y tendrás mucho más manejo de que quieres mostrar.

Tienes la posibilidad de controlar todo lo que quieras, ya sea los estilos de tu aplicación y los componentes que quieres mostrar y reutilizar en diferentes partes, además nunca había sido tan fácil manejar los estilos de tu app, te dejo un breve ejemplo:

Figura 2

Ejemplo de composable

```
@Composable
fun GreenTheme(
    darkTheme: Boolean = isSystemInDarkTheme(), //Función dada por Compose
    content: @Composable () -> Unit
) {
    val colors = if (darkTheme) {
        GreenThemeDark //Paleta de colores dark
    } else {
        GreenThemeLight //Paleta de colors light
    }
    PlatziTheme(darkTheme, colors, content)
}
```

Nota. Creación propia (2026).

Ventajas de Jetpack Compose

Menos código

Escribir menos código afecta a todas las etapas de desarrollo: como autor, puedes enfocarte en el problema en cuestión, dado que debes realizar menos pruebas y depuraciones, y se reduce el margen de error; como revisor o compilador, tienes menos código para leer, comprender, revisar y mantener.

Compose te permite realizar más tareas con menos código en comparación con el sistema Android View. Lo que sea que debas compilar (botones, listas o animaciones) ahora requiere menos código. A continuación, te mostramos algunos comentarios de nuestros socios:

- "*Para la misma clase de botón, [el código] era 10 veces más pequeño*" (Twitter).
- "*También se ve una reducción significativa a la hora de compilar cualquier pantalla con RecyclerView, que es el caso de la mayoría de nuestras pantallas*" (Monzo).
- "*Nos complace ver cuán pocas líneas se necesitan para crear listas o animaciones en nuestra app. Escribimos menos líneas de código por función, lo que nos permite enfocarnos más en ofrecer valor a nuestros clientes*" (Cuvva).

El código solamente se escribe en Kotlin, en lugar de dividirlo entre Kotlin y XML: "*Es mucho más fácil revisar el código cuando está escrito en el mismo lenguaje y, a veces, en el mismo archivo, sin necesidad de alternar entre Kotlin y XML*" (Monzo).

El código escrito con Compose es simple y fácil de mantener sin importar lo que compiles. "*El sistema de diseño de Compose es conceptualmente más sencillo, por lo que se comprende con mayor facilidad. También resulta más fácil leer el código de componentes complejos*" (Square).

Intuitivo

Compose usa una API declarativa, lo que significa que solo debes describir tu IU, y Compose se encarga del resto. Las API son intuitivas, fáciles de descubrir y usar: "*Nuestra capa de temas es mucho más intuitiva y legible. En un solo archivo de Kotlin, pudimos lograr lo que hubiese requerido muchos archivos en formato XML para definir los atributos y las asignaciones a través de varias superposiciones de temas en capas*" (Twitter).

Compose te permite compilar componentes pequeños y sin estado que no están vinculados a una actividad o un fragmento específicos. Eso hace que sean fáciles de reutilizar y probar: "*Nos propusimos ofrecer un conjunto nuevo de componentes de la IU sin estado que sean fáciles de usar y mantener, y se puedan implementar, ampliar y personalizar de manera intuitiva. Compose nos brindó una excelente respuesta*" (Twitter).

En Compose, los estados son explícitos y se pasan al elemento componible. De esa manera, hay una única fuente de información del estado, lo cual lo mantiene separado y encapsulado. Por lo tanto, a medida que cambia el estado de la app, tu IU se actualiza automáticamente. "*Ofrece un razonamiento más sencillo y una reducción de comportamientos fuera de tu control o propensos a una mala interpretación"* (Cuvva).

Acelera el desarrollo

Compose es compatible con todo tu código existente, por lo que puedes llamar código de Compose desde Views, y viceversa. Las bibliotecas más comunes como Navigation, ViewModel y las corrutinas de Kotlin funcionan con Compose, de modo que puedes comenzar a implementarlo cuando y donde quieras. "*Comenzamos a integrar Compose desde la interoperabilidad y así descubrimos que es un éxito. Descubrimos que no teníamos que pensar en cuestiones como los modos claro y oscuro, por lo que la experiencia fue impecable*" (Cuvva).

Gracias a la compatibilidad completa con Android Studio, que brinda funciones como las versiones de vista previa en vivo, podrás iterar y enviar código más rápido: "*Las vistas previas de Android Studio nos ahorraron mucho tiempo. Poder compilar varias vistas previas también nos permite ahorrar tiempo. Por lo general, debemos verificar un componente de la IU en diferentes estados o con diferentes configuraciones, como estados de error, un tamaño de fuente diferente, etc. La capacidad de crear múltiples vistas previas nos permite comprobarlo fácilmente*" (Square).

Poderoso

Compose te permite crear apps atractivas con acceso directo a las API de la plataforma de Android y compatibilidad integrada para Material Design, Tema oscuro, animaciones y mucho más: "*Compose ofrece más que una IU declarativa, también brinda API de accesibilidad, diseño y muchas mejoras. Puedes realizar lo que deseas en menos pasos*" (Square).

Con Compose, puedes brindarles vida y movimiento a tus apps con animaciones de forma rápida y fácil: "*Las animaciones son tan fáciles de agregar en Compose que no hay excusa para no animar elementos, como cambios de color, tamaño y elevación*" (Monzo). "*Puedes crear animaciones sin requisitos especiales: es similar a presentar una pantalla estática.*" (Square).

Desventajas de Jetpack Compose

Curva de aprendizaje

Adoptar Jetpack Compose requiere aprender nuevos conceptos y paradigmas, especialmente para desarrolladores acostumbrados a marcos de interfaz de usuario imperativos como los diseños basados en XML. La curva de aprendizaje inicial puede ser pronunciada, sobre todo para quienes no están familiarizados con la programación declarativa.

Documentación y soporte comunitario limitados

A pesar de su creciente popularidad, Jetpack Compose es relativamente nuevo en comparación con los frameworks de interfaz de usuario tradicionales de Android, como los diseños XML. Por lo tanto, la documentación y el soporte de la comunidad para Compose pueden ser menos extensos, lo que dificulta encontrar soluciones a problemas específicos.

Consideraciones de rendimiento

Si bien Jetpack Compose está diseñado para optimizar el rendimiento, las interfaces de usuario complejas con componentes componibles anidados o diseños ineficientes aún pueden afectar el rendimiento de la aplicación. Los desarrolladores deben diseñar cuidadosamente sus interfaces de usuario para garantizar un rendimiento de renderizado y una capacidad de respuesta óptimos.

Descripción del flujo implementado y decisiones de diseño

El flujo de la aplicación fue diseñado siguiendo una estructura lineal y clara, orientada a guiar al usuario de forma intuitiva desde el inicio de sesión hasta la visualización de resultados, se implementó un flujo de cuatro pantallas principales: Login, Bienvenida, Ingreso de notas y resultado final

En la primera pantalla, el usuario ingresa sus credenciales mediante un formulario simple con validaciones básicas. Se asegura que el correo tenga un formato válido y que la contraseña cumpla con una longitud mínima. A nivel de diseño, se utilizó un fondo degradado en tonos azules para generar una identidad visual consistente, acompañado de una card centrada con bordes redondeados y sombra, lo que aporta profundidad y enfoque en el contenido principal. Los campos incluyen iconografía y estados visuales claros para mejorar la experiencia

Luego, al iniciar sesión correctamente, el usuario es dirigido a la pantalla de bienvenida. En esta vista se personaliza la experiencia mostrando el nombre del usuario extraído del correo electrónico. Se mantiene la misma línea visual que el login, reutilizando el fondo degradado y la tarjeta central, lo que refuerza la consistencia del diseño. Aca podemos observar un título principal, el nombre del usuario resaltado y un mensaje secundario, seguido de un botón de acción claro para continuar.

En la tercera pantalla, el usuario ingresa tres notas numéricas. Aquí se tomaron decisiones importantes tanto a nivel funcional como de UX. Se restringe la entrada únicamente a números y dentro del rango de 0 a 10, evitando errores desde el origen. Además, se implementó validación para campos vacíos antes de permitir el cálculo. Desde el punto de vista visual, se mantiene la coherencia con las pantallas anteriores, También se tomó en cuenta la superposición del teclado mediante scroll y ajuste dinámico del layout, garantizando que todos los campos sean accesibles.

Así pues en la pantalla de resultados, se presenta el promedio calculado con un formato de dos decimales. Se incorporó el uso de colores dinámicos para representar el estado del resultado (verde para aprobado, amarillo para regular y rojo para reprobado). Esto permite una interpretación rápida. Se incluyen dos acciones claras que son regresar a ingresar notas o cerrar sesión, diferenciadas visualmente mediante estilos de botón.

Conclusiones

- El desarrollo de este pequeño proyecto permitió implementar un flujo completo de aplicación móvil, abarcando desde la autenticación básica hasta el procesamiento y visualización de datos, manteniendo coherencia tanto a nivel funcional como visual, al aprender a usarlo se logró construir una experiencia de usuario clara, intuitiva y alineada con estándares modernos de diseño, destacando el uso de jerarquía visual, consistencia en estilos y retroalimentación inmediata al usuario mediante validaciones
- A nivel técnico, el uso de Kotlin facilitó la escritura de código más limpio, conciso y seguro en comparación con enfoques tradicionales.
- Jetpack Compose representó un cambio importante en la forma de desarrollar interfaces, adoptando un enfoque declarativo similar a frameworks modernos de frontend
- Conceptos como composables, control de estado y reutilización de componentes se integran de forma natural, haciendo que el desarrollo sea mantenible

Conclusiones personales

Melvin Robles

Como desarrollador frontend especializado en Angular y React, esta experiencia con Kotlin y Jetpack Compose fue bastante interesante y positiva. Al inicio representó un cambio importante, pero rápidamente encontré similitudes con el desarrollo web moderno, especialmente en el enfoque declarativo, conceptos como el manejo de estado, la construcción de interfaces mediante funciones y la actualización reactiva de la UI me resultaron familiares, lo que facilitó mucho el proceso de aprendizaje.

Pude trasladar mentalmente varios conocimientos, como la estructura de componentes, el manejo de eventos y la lógica de formularios, hacia Compose. Esa capacidad de “traducir” lo que ya conocía a un nuevo entorno fue clave para adaptarme rápidamente, una vez entendida esa relación, el desarrollo se volvió mucho más fluido y natural y seguramente seguiré practicando para descubrir más de lo que se puede lograr.

Juan Carlos Ramírez

Como conclusión personal fue muy interesante formar parte de la investigación sobre Jetpack Compose al igual al ponerlo en práctica con los integrantes del grupo en la elaboración del App de Notas ya que fue muy enriquecedor. AL inicio fue necesario comprender la logia declarativa y también fue necesario analizar la interfaz de Android Studio, pero también tiene muchas cosas similares a otros programas que se han utilizado dentro de la carrera lo cual fue de mucha ayuda.

A mi punto de vista Jetpack Compose facilita mucho la organización del código y también que los componentes hacen que el desarrollo sea más claro y ordenado.

Me quedo con la satisfacción de haber puesto en práctica una herramienta que seguramente marcará el futuro del desarrollo en Android y seguiré explorando sobre Jetpack Compose porque siento que abre muchas posibilidades para crear aplicaciones intuitivas y atractivas.

Katia María Hernández Pérez

La verdad, al principio me resistía a dejar los XML porque es a lo que uno ya está acostumbrado, pero después de probar Compose para este foro, me di cuenta de que antes perdíamos el tiempo innecesariamente. Lo que más me gustó fue no tener que estar saltando de un archivo a otro solo para cambiar un color o un margen; tener todo en Kotlin hace que el flujo de trabajo sea mucho más natural. Eso sí, me costó un poco entender cómo "piensa" el framework con el manejo de estados, pero una vez que te hace clic la cabeza, te das cuenta de que escribir menos líneas no es solo por flojera, sino porque realmente el código queda mucho más limpio y menos propenso a que se te rompa la app por una tontería.

David Roberto Ferrer Coto

Lo que me llevo de esta práctica es que, aunque Google nos venda Compose como "fácil", tiene su truco. Al principio te frustras porque quieres forzar la lógica vieja en herramientas nuevas, pero cuando logramos que las notas se validaran y los colores cambiaran sin dar mil vueltas, ahí es donde se ve el potencial. Me salvó la vida la función de *Preview* de Android

Studio; me evitó la fatiga de estar corriendo el emulador cada cinco minutos para ver si un botón se veía bien o si el fondo degradado no se había arruinado. Siento que es un alivio dejar de pelear con IDs y adaptadores de listas que solo hacían el código más pesado y difícil de leer para el resto del equipo.

Referencias

Android Developers. (2024, 19 de septiembre). *Por qué adoptar Compose*. Android Developers. https://developer.android.com/develop/ui/compose/why-adopt?hl=es-419

Komilov, Y. (2023, 23 de noviembre). *Explorando los pros y los contras de Jetpack Compose: Una guía completa*. Medium. https://medium.com/@YodgorbekKomilo/exploring-the-pros-and-cons-of-jetpack-compose-a-comprehensive-guide-5e49453c924f

Platzi. (2021, 29 de diciembre). *Android Jetpack Compose: ¿Qué es?* Platzi. https://platzi.com/blog/jetpack-compose-android/.

Román, M. (2022, 2 de noviembre). *Introducción a Jetpack Compose*. Paradigma Digital. https://www.paradigmadigital.com/dev/introduccion-jetpack-compose/.

Link de video

https://youtu.be/G951tKYRbaQ?si=bOZrQor8V604uVnX